CHAPTER 6 – EVALUATING A SOLUTION

True/False Questions

1. Most real-world software solutions are proven correct through mathematical proofs rather than empirical testing.
2. An elegant solution prioritizes complexity over simplicity and effectiveness.
3. A usability test should be conducted by guiding users through the system step by step.
4. A solution that passes some tests but fails at least one test is still considered correct.
5. Big-O notation helps classify algorithms based on their worst-case performance.
6. A program can be considered correct even if it does not solve the original problem it was designed for.
7. Efficiency is only concerned with execution time, not memory usage.
8. An elegant solution is always the most complex one.
9. Usability testing should involve real users performing predefined tasks.
10. A solution with $O(N^2)$ complexity is generally more efficient than one with $O(N)$ complexity.
11. A correct solution can still be inefficient.
12. A usability evaluation can include observing users as they interact with a system and gathering their feedback.
13. Trade-offs are necessary in software development because improving one aspect may negatively affect another.

Multiple Choice Questions

1 Which of the following are key measures for evaluating a solution?
A) Correctness
B) Efficiency
C) Elegance
D) Usability

2 What does Big-O notation describe?
A) The maximum memory an algorithm will use
B) The running time growth of an algorithm
C) The number of bugs in a program
D) The usability of an algorithm

3 Which of these is **not** a component of usability?
A) Learnability
B) Flexibility
C) Memorability
D) Satisfaction

4 What is the primary goal of testing in solution evaluation?
A) To prove the program is error-free
B) To identify whether the solution is incorrect
C) To ensure users enjoy using the program

D) To make the code look elegant

5 Which of the following complexity classes describes an algorithm that always runs in a constant amount of time?
A) O(N)
B) O(log N)
C) O(1)
D) $O(N^2)$

6 Which of the following can negatively impact a solution's usability?
A) A complex user interface
B) Poor error handling
C) A long learning curve
D) All of the above

7 In Big-O notation, which class represents an algorithm that doubles in runtime as the input size doubles?
A) O(1)
B) O(N)
C) $O(N^2)$
D) O(log N)

Matching Exercise

Match each concept with its correct description:

Concept	**Description**
Correctness	Determines if the solution actually solves the intended problem.
Efficiency	Evaluates the time and space complexity of the solution.
Elegance	Measures how well the solution balances simplicity and effectiveness.
Usability	Assesses how easy and satisfying the solution is for users.
Trade-offs	The balance between different factors, such as speed vs. memory usage.
Empirical testing	Running a set of tests to check correctness in practice
Big-O Notation	A way to classify algorithm efficiency based on worst-case performance
Trade-offs	Choosing to prioritize one aspect of a solution over another
Usability Testing	Observing how users interact with a system to measure effectiveness

Concept	Description
Efficiency	Evaluating time and space complexity of an algorithm

TEXT COMPREHENSION QUESTIONS FOR CLASSROOM DISCUSSION

General Understanding

1. What are the four key quality measures of a solution's effectiveness?
2. Why is correctness considered the most important factor in evaluating a solution?
3. How does empirical testing help determine the correctness of a solution?

Critical Thinking & Analysis

4. The chapter states that "a solution is rarely correct before testing and debugging." Why do you think this is the case?
5. How does Big-O notation help in determining the efficiency of an algorithm? Can you give an example of two algorithms with different complexity and compare their efficiency?
6. The text describes elegance as a balance between simplicity and effectiveness. Can you think of a real-world example of an elegant solution (not necessarily in computing)?
7. Usability testing often includes observational studies and user feedback. Why is it important to include real users in this process rather than relying solely on the developer's perspective?

Application-Based Questions

8. Imagine you are designing a new mobile app. How would you evaluate its usability? What steps would you take to ensure it is user-friendly?
9. The chapter discusses trade-offs in solution evaluation. Can you think of a situation where you had to make a trade-off between efficiency and usability? How did you decide which aspect was more important?
10. If you were to explain the concept of complexity classes (e.g., O(1), O(N), O(N^2)) to someone with no programming background, how would you do it?